FE 680 finals

om

2026-05-10

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

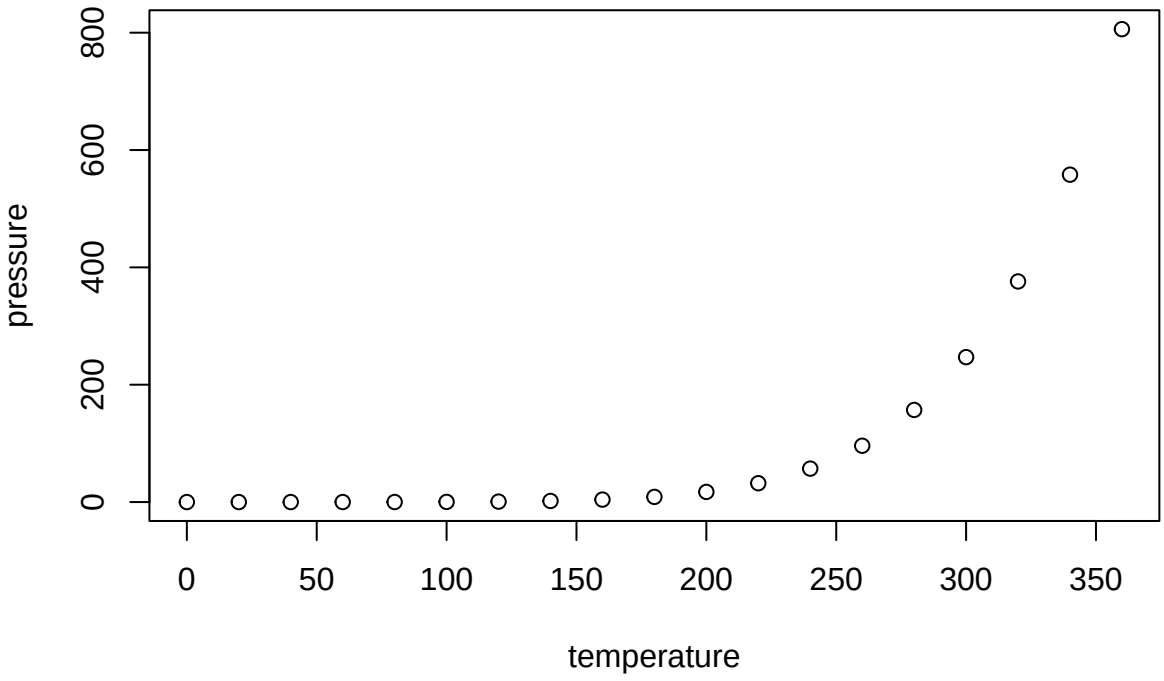
When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
summary(cars)
```

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Including Plots

You can also embed plots, for example:



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

```r
options(digits = 10)
# Problem 1: Vasicek and CIR Zero-Coupon Bond Prices/Yields

a = 0.015
b = 0.010
sigma = 0.025
r0 = 0.0257

T_values = 1:10

vasicek_price = function(T, a, b, sigma, r0) {
  B = (1 - exp(-a * T)) / a
  logA_part1 = b - sigma^2 / (2 * a^2)
  logA_part2 = B - T
  logA_part3 = sigma^2 * B^2 / (4 * a)
  logA = logA_part1 * logA_part2 - logA_part3
  A = exp(logA)
  price = A * exp(-B * r0)
  return(price)
}


cir_price = function(T, a, b, sigma, r0) {
  gamma = sqrt(a^2 + 2 * sigma^2)
  exp_gamma_T = exp(gamma * T)
  denominator = (gamma + a) * (exp_gamma_T - 1) + 2 * gamma
  B = 2 * (exp_gamma_T - 1) / denominator
  A_numerator = 2 * gamma * exp((a + gamma) * T / 2)
  A_denominator = denominator
  A_power = 2 * a * b / sigma^2
  A = (A_numerator / A_denominator)^A_power
  price = A * exp(-B * r0)
  return(price)
}


vasicek_prices = sapply(T_values, vasicek_price, a = a, b = b, sigma = sigma, r0 = r0)
cir_prices = sapply(T_values, cir_price, a = a, b = b, sigma = sigma, r0 = r0)
vasicek_yields = -log(vasicek_prices) / T_values
cir_yields = -log(cir_prices) / T_values
problem1_table = data.frame(
  T = T_values,
  Vasicek_Price = vasicek_prices,
  CIR_Price = cir_prices,
  Vasicek_Yield = vasicek_yields,
  CIR_Yield = cir_yields
)

print("Problem 1 Results:")
```

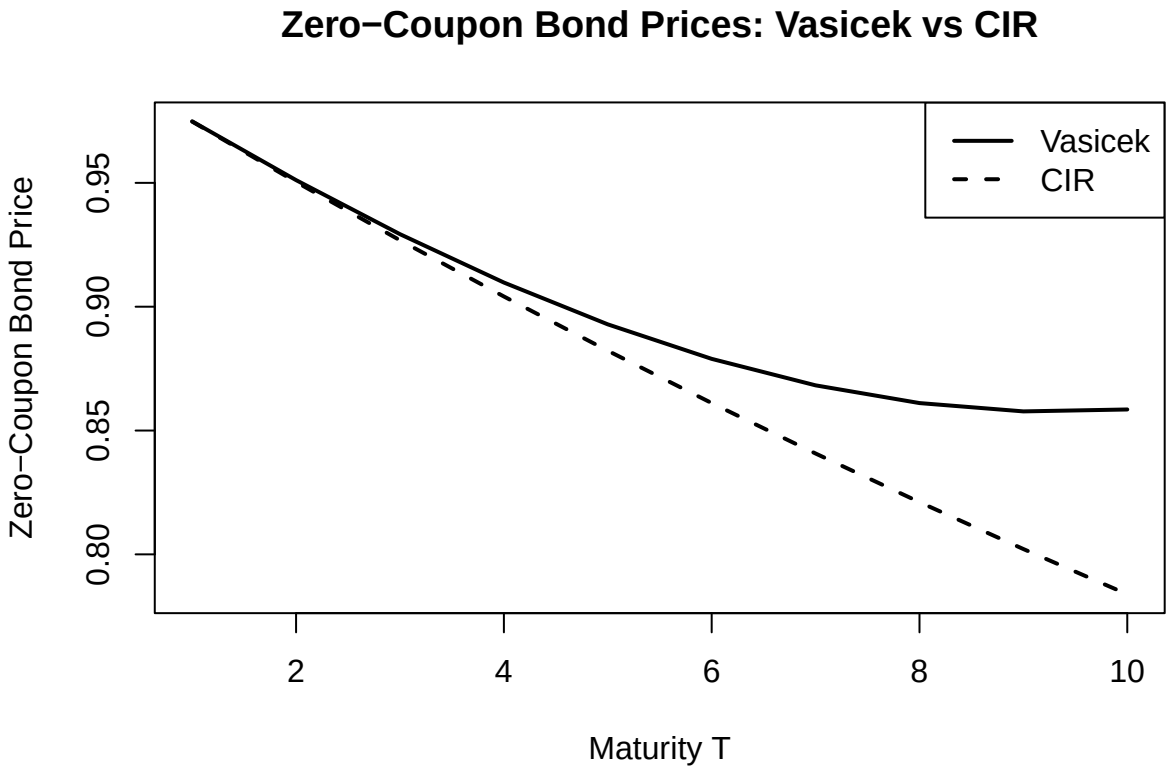
```
## [1] "Problem 1 Results:"
```

```r
print(problem1_table)
```

```
##     T Vasicek_Price    CIR_Price Vasicek_Yield     CIR_Yield
## 1   1  0.9748420378 0.9747442055 0.02547983360 0.02558019576
## 2   2  0.9511163949 0.9503615009 0.02505941592 0.02545641975
## 3   3  0.9292881378 0.9268285958 0.02444547638 0.02532887751
## 4   4  0.9097565600 0.9041218852 0.02364455793 0.02519777474
## 5   5  0.8928692639 0.8822175626 0.02266302196 0.02506331675
## 6   6  0.8789367691 0.8610917247 0.02150705316 0.02492570792
## 7   7  0.8682473417 0.8407204653 0.02018266414 0.02478515114
## 8   8  0.8610819490 0.8210799614 0.01869570003 0.02464184740
## 9   9  0.8577294241 0.8021465498 0.01705184287 0.02449599526
## 10 10  0.8585020938 0.7838967961 0.01525661597 0.02434779049
```

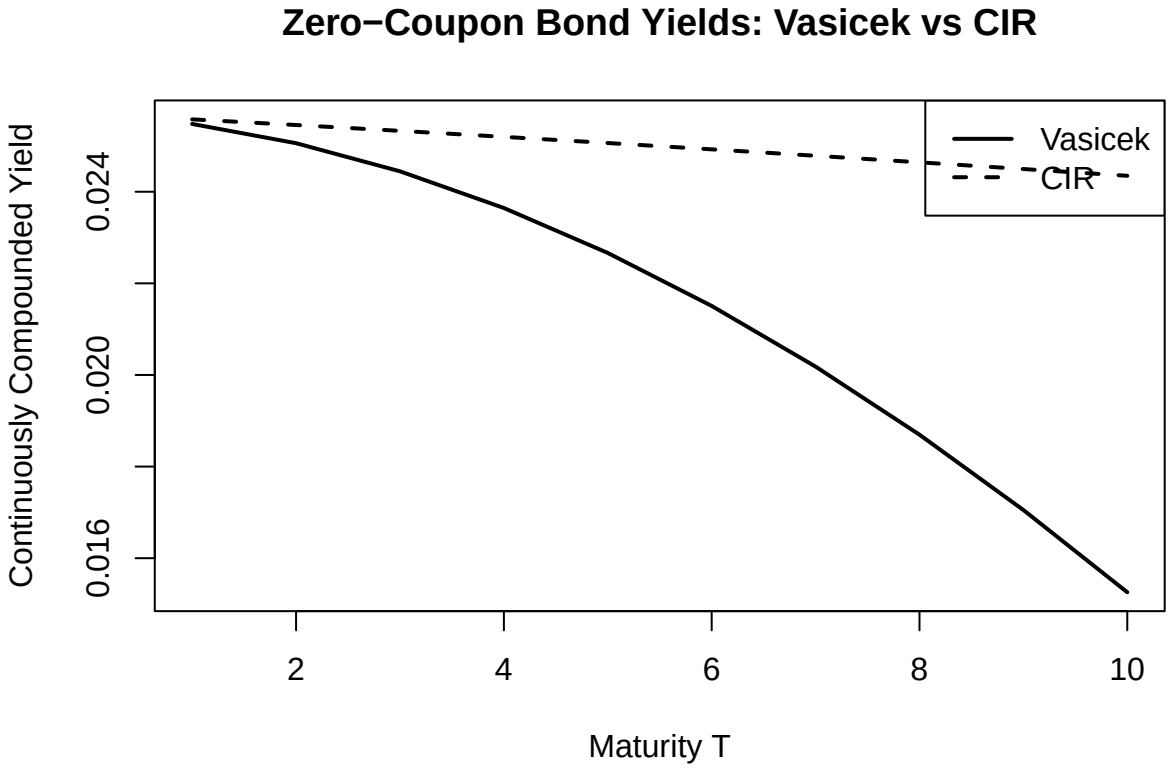
```r
# Plot bond prices

plot(
  T_values,
  vasicek_prices,
  type = "l",
  lwd = 2,
  xlab = "Maturity T",
  ylab = "Zero-Coupon Bond Price",
  main = "Zero-Coupon Bond Prices: Vasicek vs CIR",
  ylim = range(c(vasicek_prices, cir_prices)))
lines(
  T_values,
  cir_prices,
  lwd = 2,
  lty = 2)
legend(
  "topright",
  legend = c("Vasicek", "CIR"),
  lwd = 2,
  lty = c(1, 2))
```




```r
# Plot bond yields

plot(
  T_values,
  vasicek_yields,
  type = "l",
  lwd = 2,
  xlab = "Maturity T",
  ylab = "Continuously Compounded Yield",
  main = "Zero-Coupon Bond Yields: Vasicek vs CIR",
  ylim = range(c(vasicek_yields, cir_yields)))
lines(
  T_values,
  cir_yields,
  lwd = 2,
  lty = 2)
legend(
  "topright",
  legend = c("Vasicek", "CIR"),
  lwd = 2,
  lty = c(1, 2))
```




```r
# Problem 2

T = 3
F0 = 0.03572
sigma_S = 0.12
sigma_X = 0.10
rho = 0.3

r_usd = 0.038
r_jpy = 0.015

N_usd = 5000000
N_jpy = 1000000

P_usd = (1 + r_usd / 2)^(-2 * T)
P_jpy = (1 + r_jpy / 2)^(-2 * T)

# Simple expected swap rate
expected_swap_rate = F0

# Part (a)
V_usd = N_usd * P_usd * expected_swap_rate

# Part (b)
quanto_adjustment = exp(-rho * sigma_S * sigma_X * T)

V_jpy = N_jpy * P_jpy * expected_swap_rate * quanto_adjustment

print("Problem 2(a): Dollar Value:")
```

```
## [1] "Problem 2(a): Dollar Value:"
```

```
print(V_usd)
```

```
## [1] 159527.7906
```

```
print("Quanto Adjustment:")
```

```
## [1] "Quanto Adjustment:"
```

```
print(quanto_adjustment)
```

```
## [1] 0.9892581106
```

```
print("Problem 2(b): Yen Value:")
```

```
## [1] "Problem 2(b): Yen Value:"
```

```
print(V_jpy)
```

```
## [1] 33787.08629
```

```
print("Problem 2 Summary:")
```

```
## [1] "Problem 2 Summary:"
```

```
print(data.frame(
  Quantity = c("USD Discount Factor", "JPY Discount Factor",
               "Expected Swap Rate", "Quanto Adjustment",
               "Dollar Value", "Yen Value"),
  Value = c(P_usd, P_jpy, expected_swap_rate, quanto_adjustment, V_usd, V_jpy)
))
```

```
##              Quantity           Value
## 1 USD Discount Factor 8.932127135e-01
## 2 JPY Discount Factor 9.561580178e-01
## 3  Expected Swap Rate 3.572000000e-02
## 4   Quanto Adjustment 9.892581106e-01
## 5        Dollar Value 1.595277906e+05
## 6           Yen Value 3.378708629e+04
```

```
# Problem 3: Hazard Rate and Recovery Rate
# Given data
r = 0.035

s_cds = 0.018
s_binary = 0.028

T_cds = 5


premium_regular = function(lambda, T_cds, r) {
  total = 0
  for (i in 1:T_cds) {
    discount_factor = exp(-r * i)
    survival_probability = exp(-lambda * i)
    present_value = discount_factor * survival_probability
    total = total + present_value
  }
  return(total)
```

```
}


premium_accrued = function(lambda, T_cds, r) {
  total = 0
  for (i in 1:T_cds) {
    survival_start = exp(-lambda * (i - 1))
    survival_end = exp(-lambda * i)
    default_probability = survival_start - survival_end
    discount_factor = exp(-r * (i - 0.5))
    accrued_payment = 0.5 * discount_factor * default_probability
    total = total + accrued_payment
  }
  return(total)
}


default_leg_binary = function(lambda, T_cds, r) {
  total = 0
  for (i in 1:T_cds) {
    survival_start = exp(-lambda * (i - 1))
    survival_end = exp(-lambda * i)
    default_probability = survival_start - survival_end
    discount_factor = exp(-r * (i - 0.5))
    default_payment_value = discount_factor * default_probability
    total = total + default_payment_value
  }
  return(total)
}

binary_spread_model = function(lambda) {
  A = premium_regular(lambda, T_cds, r)
  B = premium_accrued(lambda, T_cds, r)
  D = default_leg_binary(lambda, T_cds, r)
  spread = D / (A + B)
  return(spread)
}


objective = function(lambda) {
  model_spread = binary_spread_model(lambda)
  difference = model_spread - s_binary
  return(difference)
}

solution = uniroot(
  objective,
  lower = 0.000001,
  upper = 1
)

lambda_solution = solution$root
```

```r
R_solution = 1 - s_cds / s_binary


# Check the results

A = premium_regular(lambda_solution, T_cds, r)
B = premium_accrued(lambda_solution, T_cds, r)
D = default_leg_binary(lambda_solution, T_cds, r)
binary_spread_check = D / (A + B)
vanilla_spread_check = (1 - R_solution) * D / (A + B)
problem3_summary = data.frame(
  Quantity = c(
    "Hazard rate lambda",
    "Hazard rate lambda percent",
    "Recovery rate R",
    "Recovery rate percent",
    "Binary CDS spread check",
    "Vanilla CDS spread check"
  ),
  Value = c(
    lambda_solution,
    100 * lambda_solution,
    R_solution,
    100 * R_solution,
    binary_spread_check,
    vanilla_spread_check
  )
)

print("Problem 3 Results:")
```

```
## [1] "Problem 3 Results:"
```

```r
print(problem3_summary)
```

```
##                      Quantity          Value
## 1         Hazard rate lambda  0.02752361695
## 2 Hazard rate lambda percent  2.75236169533
## 3            Recovery rate R  0.35714285714
## 4      Recovery rate percent 35.71428571429
## 5    Binary CDS spread check  0.02800094898
## 6   Vanilla CDS spread check  0.01800061006
```

```r
# Problem 4: Hull-White Two-Factor Model
# Given parameters
a = 0.1
b = 1.4

sigma1 = 0.01
sigma2 = 0.008

rho = 0.7

r0 = 0.05
u0 = 0.01
```

```
# Maturities
T_values = 1:5

# Current time
t = 1

P0 = function(T) {
  price = exp(-0.05 * T)
  return(price)
}

P_1T = function(T) {
  price = P0(T) / P0(1)
  return(price)
}


B1_function = function(t, T, a) {
  tau = T - t
  if (tau == 0) {
    return(0)}
  B1 = (1 - exp(-a * tau)) / a
  return(B1)
}

B2_function = function(t, T, a, b) {
  tau = T - t
  if (tau == 0) {
    return(0)}
  term1 = (1 - exp(-b * tau)) / b
  term2 = (1 - exp(-a * tau)) / a
  B2 = (term1 - term2) / (a - b)
  return(B2)
}


sigma_P_function = function(t, T, a, b, sigma1, sigma2, rho) {
  B1 = B1_function(t, T, a)
  B2 = B2_function(t, T, a, b)
  variance_part1 = B1^2 * sigma1^2
  variance_part2 = B2^2 * sigma2^2
  variance_part3 = 2 * rho * B1 * B2 * sigma1 * sigma2
  variance = variance_part1 + variance_part2 + variance_part3
  sigma_P = sqrt(variance)
  return(sigma_P)
}


sigma_R_function = function(t, T, a, b, sigma1, sigma2, rho) {
  tau = T - t
  if (tau == 0) {
    return(NA)}
  sigma_P = sigma_P_function(t, T, a, b, sigma1, sigma2, rho)
```

```
  sigma_R = sigma_P / tau
  return(sigma_R)
}

P_1T_values = sapply(T_values, P_1T)
B1_values = sapply(
  T_values,
  function(T) B1_function(t, T, a))
B2_values = sapply(
  T_values,
  function(T) B2_function(t, T, a, b))
sigma_P_values = sapply(
  T_values,
  function(T) sigma_P_function(t, T, a, b, sigma1, sigma2, rho))
sigma_R_values = sapply(
  T_values,
  function(T) sigma_R_function(t, T, a, b, sigma1, sigma2, rho))

problem4_table = data.frame(
  T = T_values,
  tau = T_values - t,
  P_1T = P_1T_values,
  B1 = B1_values,
  B2 = B2_values,
  sigma_P = sigma_P_values,
  sigma_R = sigma_R_values
)

print("Problem 4 Results:")
```

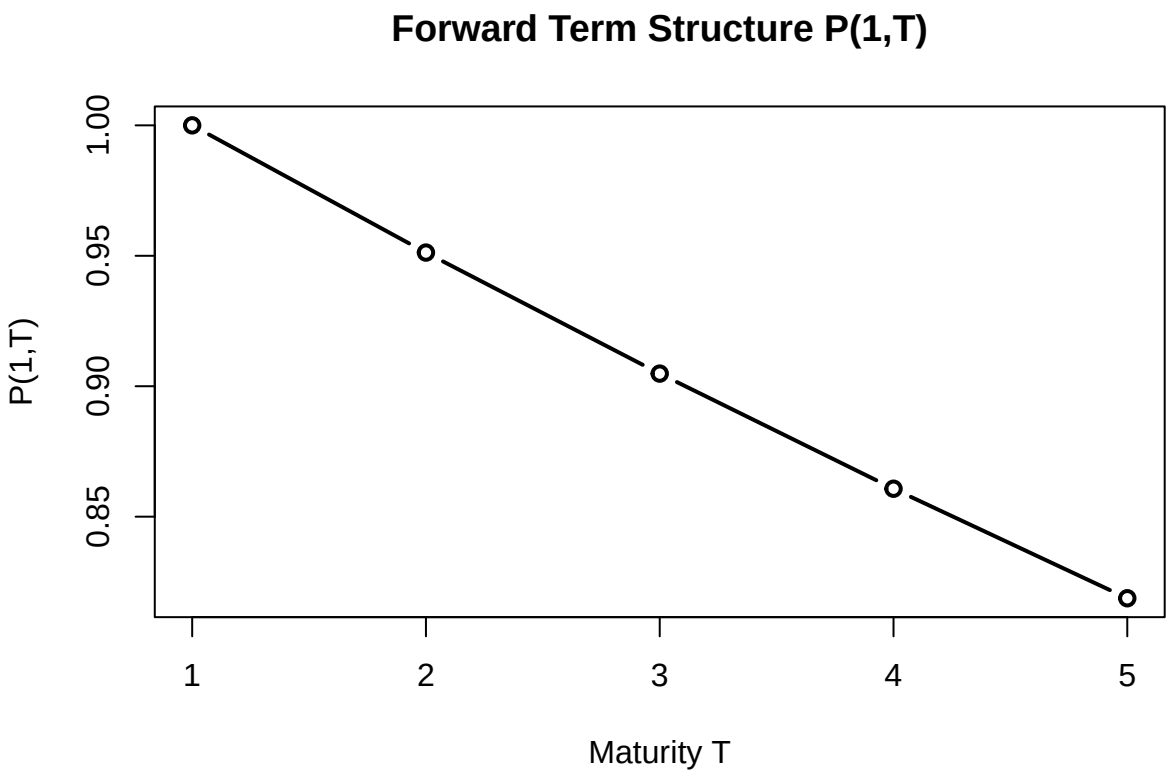
```
## [1] "Problem 4 Results:"
```

```
print(problem4_table)
```

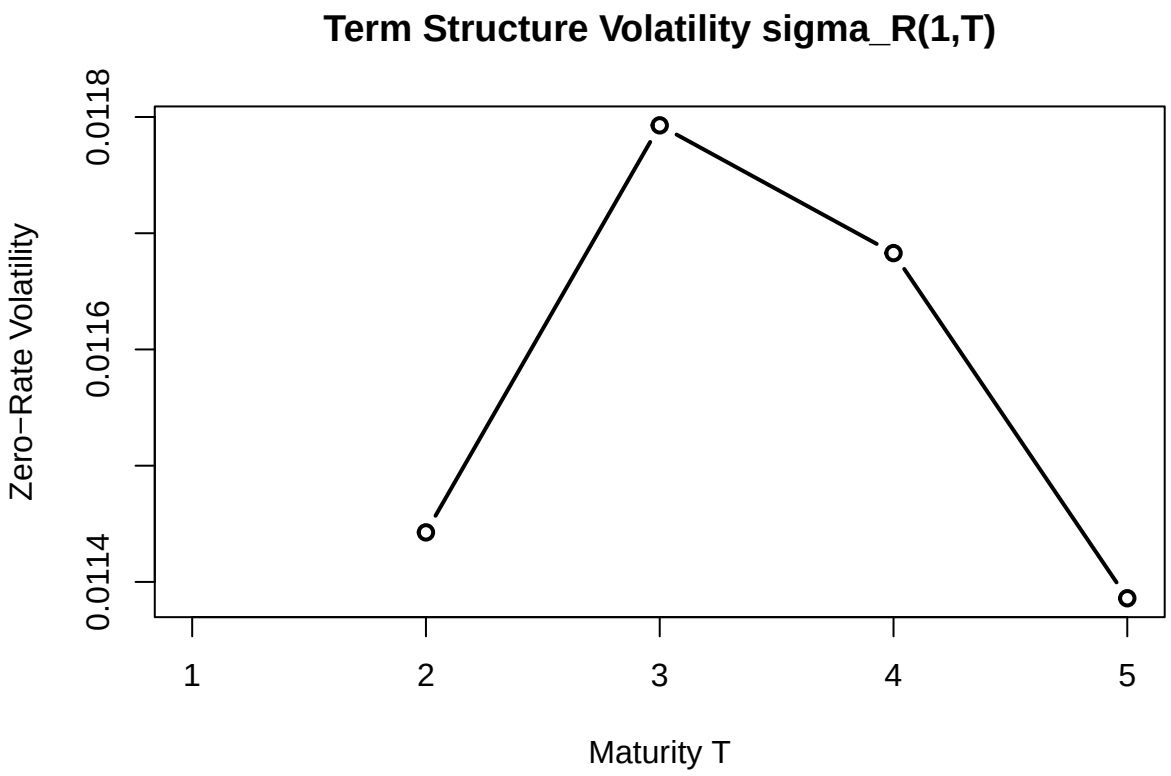
```
##   T tau         P_1T           B1           B2       sigma_P       sigma_R
## 1 1   0 1.0000000000 0.0000000000 0.0000000000 0.00000000000            NA
## 2 2   1 0.9512294245 0.9516258196 0.3180621491 0.01144261182 0.01144261182
## 3 3   2 0.9048374180 1.8126924692 0.8783403953 0.02358563697 0.01179281849
## 4 4   3 0.8607079764 2.5918177932 1.4524947732 0.03504869151 0.01168289717
## 5 5   4 0.8187307531 3.2967995396 1.9885808897 0.04554383010 0.01138595753
```

```
# Plot P(1,T)

plot(
  problem4_table$T,
  problem4_table$P_1T,
  type = "b",
  lwd = 2,
  xlab = "Maturity T",
  ylab = "P(1,T)",
  main = "Forward Term Structure P(1,T)"
)
```




```r
# Plot zero-rate volatility
plot(
  problem4_table$T,
  problem4_table$sigma_R,
  type = "b",
  lwd = 2,
  xlab = "Maturity T",
  ylab = "Zero-Rate Volatility",
  main = "Term Structure Volatility sigma_R(1,T)"
)
```




```r
#Problem 5
set.seed(123)
# Inputs
n_names = 125
n_paths = 100000
rho = 0.25
R = 0.40
LGD = 1 - R
T = 1

p = runif(n_names, min = 0.01, max = 0.20)

lambda = -log(1 - p) / T

Z = rnorm(n_paths)

portfolio_loss = numeric(n_paths)
default_count = numeric(n_paths)

for (m in 1:n_paths) {

  eps = rnorm(n_names)
  X = sqrt(rho) * Z[m] + sqrt(1 - rho) * eps
  U = pnorm(X)
  tau = -log(U) / lambda
  default_indicator = as.numeric(tau <= T)
  default_count[m] = sum(default_indicator)
  portfolio_loss[m] = LGD * default_count[m] / n_names
}
```

```r
loss_summary = quantile(
  portfolio_loss,
  probs = c(0.01, 0.05, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99)
)

print("Portfolio Loss Distribution")
```

```
## [1] "Portfolio Loss Distribution"
```

```r
print(loss_summary)
```

```
##     1%     5%    25%    50%    75%    90%    95%    99%
## 0.0000 0.0048 0.0192 0.0480 0.0864 0.1440 0.1824 0.2688
```

```r
cat("Mean portfolio loss:", mean(portfolio_loss), "\n")
```

```
## Mean portfolio loss: 0.063420528
```

```r
cat("Standard deviation:", sd(portfolio_loss), "\n")
```

```
## Standard deviation: 0.05900929901
```

```r
tranche_loss = function(L, A, D) {
  pmin(pmax(L - A, 0), D - A)
}

equity_loss = tranche_loss(portfolio_loss, 0.00, 0.03)
mezz_loss = tranche_loss(portfolio_loss, 0.03, 0.10)
senior_loss = tranche_loss(portfolio_loss, 0.10, 1.00)

# Expected losses as portfolio percentage
EL_equity = mean(equity_loss)
EL_mezz = mean(mezz_loss)
EL_senior = mean(senior_loss)

# Expected losses as tranche percentage
EL_equity_pct = EL_equity / 0.03
EL_mezz_pct = EL_mezz / 0.07
EL_senior_pct = EL_senior / 0.90

results = data.frame(
  Tranche = c("Equity", "Mezzanine", "Senior"),
  Attachment = c(0.00, 0.03, 0.10),
  Detachment = c(0.03, 0.10, 1.00),
  Expected_Loss_Portfolio_Percent = c(EL_equity, EL_mezz, EL_senior),
  Expected_Loss_Tranche_Percent = c(EL_equity_pct, EL_mezz_pct, EL_senior_pct)
)

print(results)
```

```
##     Tranche Attachment Detachment Expected_Loss_Portfolio_Percent
## 1    Equity       0.00       0.03                     0.024403884
## 2 Mezzanine       0.03       0.10                     0.027135196
## 3    Senior       0.10       1.00                     0.011881448
##   Expected_Loss_Tranche_Percent
## 1                  0.81346280000
```

```
## 2                 0.38764565714
## 3                 0.01320160889
```

```
# Problem 6(a): Cubic spline interpolation
maturity = c(0.5, 5, 10)
yield = c(0.0343, 0.0348, 0.0397)
spline_fit = splinefun(maturity, yield, method = "natural")
y3 = spline_fit(3)
cat("Interpolated 3-year yield:", y3, "\n")
```

```
## Interpolated 3-year yield: 0.03422209227
```

```
cat("Interpolated 3-year yield in percent:", 100 * y3, "%\n")
```

```
## Interpolated 3-year yield in percent: 3.422209227 %
```

```
# Problem 6(b): Duration-convexity hedge

B = function(T, y) {
  exp(-y * T)
}

B_05 = B(0.5, 0.0343)
B_5 = B(5, 0.0348)
B_10 = B(10, 0.0397)
B_3 = B(3, y3)

cat("B(0.5):", B_05, "\n")
```

```
## B(0.5): 0.9829962241
```

```
cat("B(5):", B_5, "\n")
```

```
## B(5): 0.8402968977
```

```
cat("B(10):", B_10, "\n")
```

```
## B(10): 0.6723340256
```

```
cat("B(3):", B_3, "\n")
```

```
## B(3): 0.9024280844
```

```
A = matrix(
  c(
    B_05,       B_5,       B_10,
    0.5*B_05,   5*B_5,    10*B_10,
    0.5^2*B_05, 5^2*B_5,  10^2*B_10
  ),
  nrow = 3,
  byrow = TRUE
)

b = c(
  -B_3,
  -3 * B_3,
  -3^2 * B_3
)
```

```r
x = solve(A, b)

names(x) = c("Position_in_B_0.5", "Position_in_B_5", "Position_in_B_10")

print(x)
```

```
## Position_in_B_0.5   Position_in_B_5  Position_in_B_10
##      -0.300644089      -0.835286328       0.141287554
```

```r
portfolio_value = B_3 + x[1]*B_05 + x[2]*B_5 + x[3]*B_10

first_order_sensitivity =
  -3*B_3 - 0.5*x[1]*B_05 - 5*x[2]*B_5 - 10*x[3]*B_10

second_order_sensitivity =
  3^2*B_3 + 0.5^2*x[1]*B_05 + 5^2*x[2]*B_5 + 10^2*x[3]*B_10

cat("Portfolio value:", portfolio_value, "\n")
```

```
## Portfolio value: -1.387778781e-17
```

```r
cat("First-order sensitivity:", first_order_sensitivity, "\n")
```

```
## First-order sensitivity: 6.661338148e-16
```

```r
cat("Second-order sensitivity:", second_order_sensitivity, "\n")
```

```
## Second-order sensitivity: -1.776356839e-15
```